

TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG



Hội thi Toán học – Lý thuyết và Ứng dụng năm học 2025–2026

Chủ đề: “Toán học trong kỷ nguyên trí tuệ nhân tạo”

CODE NGUỒN DỮ LIỆU
TRÍCH ĐOẠN MÃ PYTHON TÀNG TRUY XUẤT
DỮ LIỆU HOSE – ỨNG DỤNG FINSCOPE

ĐƠN VỊ: KHOA TOÁN – THỐNG KÊ

NHÓM SINH VIÊN THỰC HIỆN:

1. Nguyễn Thành Danh (Trưởng nhóm) — C2300014
2. Trần Huỳnh Nhã Trúc — C2300189

TP. Hồ Chí Minh, tháng 5 năm 2026

1. Giới thiệu

FinScope sử dụng dữ liệu thời gian thực 53 mã chứng khoán niêm yết trên Sở Giao dịch Chứng khoán Thành phố Hồ Chí Minh (HOSE) thông qua thư viện mã nguồn mở `vnstock 4.0.4`. Toàn bộ logic **lấy – làm sạch – chuẩn hoá** dữ liệu được tập trung tại thư mục `data/` của mã nguồn. Tài liệu này trích đoạn ba file cốt lõi của tầng dữ liệu để bạn giám khảo đối chiếu:

- `data/_clients.py` – singleton kết nối + bộ điều tiết tốc độ truy vấn (token bucket) dưới 20 yêu cầu/phút.
- `data/fetcher.py` – truy xuất giá lịch sử 1 mã, kèm các chỉ báo kỹ thuật cơ bản (MA, RSI).
- `data/market.py` – truy xuất snapshot thị trường (giá khớp lệnh hiện tại, vốn hoá, room nước ngoài) cho danh sách mã.

Toàn bộ phần còn lại của ứng dụng (14 trang giao diện, 8 mô hình dự báo, 4 thuật toán toán – tài chính cổ điển, hệ thống xác thực và cá nhân hoá) chạy trực tiếp trên **Streamlit Community Cloud** – bạn giám khảo truy cập qua trình duyệt mà không cần cài đặt môi trường Python (chi tiết tại tệp `Mo_ta_san_pham.pdf`).

2. `data/_clients.py` – Singleton client và token bucket

File này giải quyết hai bài toán: (i) tái sử dụng kết nối `vnstock` – khởi tạo một lần, dùng chung mọi nơi (tránh TLS handshake 200–500 ms cho mỗi lần truy vấn); (ii) chia sẻ một *token bucket* duy nhất giữa mọi luồng (foreground + warmer nền) để tổng tốc độ không vượt giới hạn của `vnstock`.

```

1  """Singleton vnstock clients (`@st.cache_resource`) + token bucket throttle
2  để mọi module trong app gọi vnstock CHIA SẺ cùng 1 client + cùng 1 rate limit.
3
4  Trước v56: mỗi `data.fetcher._fetch_raw` / `data.market.market_snapshot` /
5  `data.fundamental.fetch_financials` / `services.capm.fetch_vnindex` rebuild
6  `Vnstock()` instance fresh → TLS handshake ~200-500ms × 4 module × N call.
7
8  Sau v56: 1 `Vnstock()` root + N `vn_stock(symbol)` cached_resource (mỗi
9  symbol 1 lần init + tái dùng) + `throttle()` token bucket guarantee tổng
10 mọi thread/module ≤ 1 call / MIN_INTERVAL_SEC = 1 call / 3.4s = 17.6 r/m
11 < 20 r/m vnstock ceiling.
12
13 THAM KHẢO:
14   Streamlit docs – cache_resource pattern for connections.
15   GitHub finance/quant Streamlit repos.
16 """
17 from __future__ import annotations
18 import threading
19 import time
20
21 import streamlit as st
22
23
24 # — Token bucket throttle – chia sẻ giữa MỌI thread (preload, warmer, foreground)
25 MIN_INTERVAL_SEC = 3.4    # warmer/background: 17.6 req/min < 20 ceiling
26 MIN_INTERVAL_FG_SEC = 1.6 # foreground:          up to 37 r/m worst-case; thực
27                             # tế chỉ chậm khi user spam ticker. Tradeoff:
28                             # bỏ freeze 3.4s khi đổi mã lần đầu.
29 _LAST_CALL = [0.0]
30 _LOCK = threading.Lock()
31 # Foreground priority: khi user click ticker mới, set tới deadline → warmer

```

```

32 # nhường nhịn (đợi tới đó mới xin token). Bỏ race-condition fetch ngắt nhau.
33 _FG_PRIO_UNTIL = [0.0]
34
35
36 def throttle(min_interval: float = MIN_INTERVAL_SEC,
37             is_foreground: bool = False) -> None:
38     """Block tới khi đảm bảo cách lần API call trước  $\geq$  `min_interval`.
39
40     Thread-safe. Tất cả vnstock-touching code phải gọi throttle() ngay
41     trước khi gọi `.quote.history()` / `.price_board()` / `.finance.*`.
42
43     Khi `is_foreground=True`: dùng `MIN_INTERVAL_FG_SEC` (ngắn hơn) +
44     đặt nhân ưu tiên – warmer/background gặp nhân này sẽ đợi cho tới khi
45     foreground claim xong slot. UX: bỏ giạt khi user đổi mã.
46
47     AppTest / pytest detect  $\rightarrow$  bypass để không block test suite.
48     """
49     import os
50     if os.environ.get('STREAMLIT_TEST') == '1' or os.environ.get('PYTEST_CURRENT_TEST'):
51         return
52     if is_foreground:
53         min_interval = MIN_INTERVAL_FG_SEC
54         _FG_PRIO_UNTIL[0] = time.time() + 0.8 # foreground giữ ưu tiên 0.8s
55     else:
56         # Background/warmer: nhường foreground nếu đang trong giai đoạn ưu tiên.
57         while time.time() < _FG_PRIO_UNTIL[0]:
58             time.sleep(0.2)
59     with _LOCK:
60         dt = time.time() - _LAST_CALL[0]
61         if dt < min_interval:
62             time.sleep(min_interval - dt)
63         _LAST_CALL[0] = time.time()
64
65
66 def throttle_fg() -> None:
67     """Tiện ích cho code foreground (user-initiated fetch)."""
68     throttle(is_foreground=True)
69
70
71 @st.cache_resource(show_spinner=False)
72 def vn_root():
73     """Vnstock() root – singleton trên toàn process. Init 1 lần, share."""
74     from vnstock import Vnstock
75     return Vnstock()
76
77
78 @st.cache_resource(show_spinner=False, max_entries=128)
79 def vn_stock(symbol: str, source: str = 'VCI'):
80     """Stock handle cho 1 ticker – cached_resource theo (symbol, source).
81     Mỗi ticker chỉ init 1 lần / process, không phải mỗi call.
82     """
83     return vn_root().stock(symbol=symbol, source=source)
84
85
86 @st.cache_resource(show_spinner=False)
87 def vn_trading(source: str = 'VCI'):
88     """Trading handle cho market snapshot – singleton."""
89     from vnstock import Trading
90     return Trading(source=source)

```

3. data/fetcher.py – Truy xuất giá lịch sử và chỉ báo

Hàm `fetch_data(ticker)` là cửa ngõ chính để mọi tầng phía trên (mô hình, biểu đồ, bộ máy tín hiệu) lấy chuỗi giá. Kết quả được cache đĩa 24 giờ thông qua `@st.cache_data(persist="disk")`; các chỉ báo cơ bản (MA5/20/50, MA5_Vol, RSI14) được tính ngay tại tầng dữ liệu để mọi page tái sử dụng.

```

1 import streamlit as st
2 import pandas as pd
3 import numpy as np
4 from core.config import DATA_START, DATA_END, DATA_SOURCE, CACHE_TTL
5
6
7 @st.cache_data(ttl=CACHE_TTL, show_spinner=False, persist="disk")
8 def _fetch_raw(ticker: str) -> pd.DataFrame:
9     """Fetch raw từ vnstock + indicator technical. Cache theo ticker (không
10     phụ thuộc date_from/date_to) → đổi date range KHÔNG gọi lại network.
11
12     Bọc redirect_stdout(StringIO) để NUỐT các thông báo tiếng Việt vnstock
13     spam ra console (quảng cáo Insiders + cảnh báo update). Trên Windows
14     cp1252 các ký tự đó gây UnicodeEncodeError → vờ fetch (đã thấy ở ACB).
15     """
16     import contextlib, io
17     # v56 – Dùng singleton stock handle + throttle để stay dưới vnstock 20 r/m.
18     # v57 – gọi từ user-initiated fetch → throttle_fg() (1.6s thay vì 3.4s).
19     from data._clients import vn_stock, throttle_fg
20     with contextlib.redirect_stdout(io.StringIO()), contextlib.redirect_stderr(io.StringIO()):
21         throttle_fg()
22         s = vn_stock(ticker, DATA_SOURCE)
23         df = s.quote.history(start=DATA_START, end=DATA_END, interval='1D')
24         df = df.rename(columns={
25             'time': 'Ngay', 'close': 'Close', 'open': 'Open',
26             'high': 'High', 'low': 'Low', 'volume': 'Volume',
27         })
28         df['Ngay'] = pd.to_datetime(df['Ngay']).dt.date
29         df = df.sort_values('Ngay').reset_index(drop=True)
30
31         # Technical indicators
32         df['MA5'] = df['Close'].rolling(5).mean()
33         df['MA20'] = df['Close'].rolling(20).mean()
34         df['MA50'] = df['Close'].rolling(50).mean()
35         df['MA5_Vol'] = df['Volume'].rolling(5).mean()
36
37         delta = df['Close'].diff()
38         gain = delta.clip(lower=0).rolling(14).mean()
39         loss = (-delta.clip(upper=0)).rolling(14).mean()
40         df['RSI14'] = 100 - 100 / (1 + gain / loss.replace(0, np.nan))
41         # Giá đứng yên 14 phiên (gain==loss==0) → RSI = NaN. dropna() sau đó
42         # sẽ XÓA SẠCH phiên đầu chart cho mã thanh khoản kém → silent data loss.
43         # Fillna 50 (trung tính: không tăng/giảm) → giữ phiên, dùng được L1 lag.
44         df['RSI14'] = df['RSI14'].fillna(50.0)
45
46         df['MA5_ratio'] = (df['Close'] / df['MA5'] - 1) * 100
47         df['MA20_ratio'] = (df['Close'] / df['MA20'] - 1) * 100
48         df['Volume_ratio'] = df['Volume'] / df['MA5_Vol']
49         df['Range'] = df['High'] - df['Low']
50         df['Range_ratio'] = (df['High'] - df['Low']) / df['Close'] * 100
51         df['Return'] = df['Close'].pct_change() * 100
52
53         # Lag-1 features (for MLR and CART)
54         df['Close_L1'] = df['Close'].shift(1)

```

```

55 df['Volume_L1'] = df['Volume'].shift(1)
56 df['Range_L1'] = df['Range'].shift(1)
57 df['Return_L1'] = df['Return'].shift(1)
58 df['Volume_ratio_L1'] = df['Volume_ratio'].shift(1)
59 df['Range_ratio_L1'] = df['Range_ratio'].shift(1)
60 df['MA5_ratio_L1'] = df['MA5_ratio'].shift(1)
61 df['MA20_ratio_L1'] = df['MA20_ratio'].shift(1)
62 df['RSI14_L1'] = df['RSI14'].shift(1)
63
64 df = df.dropna().reset_index(drop=True)
65
66 # Guard: nếu phiên cuối là HÔM NAY và HOSE chưa đóng cửa (trước 15:00 VN),
67 # loại bỏ hàng cuối vì Close là giá intraday chưa chốt chính thức.
68 import datetime as _dt
69 try:
70     from zoneinfo import ZoneInfo
71     _now_vn = _dt.datetime.now(ZoneInfo('Asia/Ho_Chi_Minh'))
72 except Exception:
73     _now_vn = _dt.datetime.utcnow() + _dt.timedelta(hours=7)
74
75 if len(df) > 0:
76     _last_row_date = df['Ngày'].iloc[-1]
77     _today_vn = _now_vn.date()
78     # HOSE đóng 14:45; buffer tới 15:00 để chắc chắn API đã cập nhật giá chốt
79     _close_cutoff = _dt.time(15, 0)
80     if _last_row_date == _today_vn and _now_vn.time() < _close_cutoff:
81         df = df.iloc[:-1].reset_index(drop=True)
82
83 return df
84
85
86 def fetch_data(ticker: str, date_from=None, date_to=None) -> pd.DataFrame:
87     """Public fetcher – lấy raw cache + filter theo date range (in-memory)."""
88     df = _fetch_raw(ticker)
89     if date_from:
90         df = df[df['Ngày'] >= date_from]
91     if date_to:
92         df = df[df['Ngày'] <= date_to]
93     return df.reset_index(drop=True)

```

4. data/market.py – Snapshot toàn thị trường

Hàm `market_snapshot(tickers)` dùng cho các trang Tổng quan thị trường, Phân tích cơ bản và Dashboard. Một lần truy vấn `price_board` của `vnstock` trả về dữ liệu cho cả danh sách 53 mã, tiết kiệm thời gian so với truy vấn từng mã một.

```

1  """Tổng quan Thị trường – snapshot 53 mã HOSE: giá, % thay đổi, market cap.
2
3  Nguồn: vnstock Trading.price_board (1 lệnh gọi cho cả 53 mã → nhẹ hơn fetch
4  từng mã). Cache 5 phút để không hammer API.
5
6  Đây là dữ liệu CƠ BẢN nhất cho app chuyên chứng khoán – thiếu cái này
7  thì không thấy bức tranh thị trường, chỉ thấy 1 mã đang xem.
8  """
9  import streamlit as st
10 import pandas as pd
11 import numpy as np
12

```

```

13 from core.constants import TICKERS, TICKER_INFO
14
15
16 def _sector_of(tk: str) -> str:
17     """Ngành rút gọn (bỏ '. HOSE')."""
18     s = TICKER_INFO.get(tk, '')
19     return s.split('.')[0].strip() if s else 'Khác'
20
21
22 @st.cache_data(ttl=300, show_spinner=False)
23 def market_snapshot(symbols: tuple) -> pd.DataFrame:
24     """Snapshot toàn bộ symbols → DataFrame:
25     ['ticker', 'sector', 'ref_price', 'last_price', 'change', 'change_pct',
26     'volume', 'value_M', 'market_cap_B', 'listed_share'].
27
28     Giá ở đơn vị đồng (raw từ price_board). value_M = triệu đ; market_cap_B =
29     tỷ đ.
30     """
31     import contextlib, io
32     # v56 – Dùng singleton + throttle (chia rate-limit với fetcher/fundamental/capm)
33     from data._clients import vn_trading, throttle
34     # — Fail-safe: nếu VCI Trading down hoặc trả None/schema lạ → DataFrame
35     # rỗng có schema đúng. CACHE sẽ giữ failure 5 phút → tránh dồn retry,
36     # nhưng app KHÔNG vỡ (các trang gọi market_snapshot sẽ thấy empty df).
37     _empty_cols = ['ticker', 'sector', 'ref_price', 'last_price', 'change',
38                   'change_pct', 'volume', 'value_M', 'market_cap_B', 'listed_share']
39     try:
40         with contextlib.redirect_stdout(io.StringIO()),
41             ↪ contextlib.redirect_stderr(io.StringIO()):
42             throttle()
43             pb = vn_trading('VCI').price_board(symbols_list=list(symbols))
44             if pb is None or len(pb) == 0:
45                 return pd.DataFrame(columns=_empty_cols)
46     except Exception:
47         return pd.DataFrame(columns=_empty_cols)
48     rows = []
49     for _, r in pb.iterrows():
50         try:
51             tk = str(r[('listing', 'symbol')])
52             ref = float(r[('listing', 'ref_price')] or 0)
53             shares = float(r[('listing', 'listed_share')] or 0)
54             match = float(r[('match', 'match_price')] or 0) or ref
55             vol = float(r[('match', 'accumulated_volume')] or 0)
56             val = float(r[('match', 'accumulated_value')] or 0)
57             chg = match - ref
58             pct = (chg / ref * 100) if ref > 0 else 0.0
59             mcap_billion = match * shares / 1e9 # đồng → tỷ đồng
60             rows.append({
61                 'ticker': tk,
62                 'sector': _sector_of(tk),
63                 'ref_price': ref,
64                 'last_price': match,
65                 'change': chg,
66                 'change_pct': pct,
67                 'volume': vol,
68                 'value_M': val / 1e3, # input đã ×1000 → MM đồng
69                 'market_cap_B': mcap_billion,
70                 'listed_share': shares,
71             })
72         except Exception:
73             continue

```

```

73     df = pd.DataFrame(rows)
74     return df
75
76
77 def market_kpis(df: pd.DataFrame) -> dict:
78     """KPI tổng hợp toàn thị trường mẫu."""
79     if df.empty:
80         return {'n': 0, 'n_up': 0, 'n_down': 0, 'n_flat': 0,
81                 'avg_pct': 0.0, 'total_mcap_T': 0.0, 'total_value_B': 0.0}
82     n_up = int((df['change_pct'] > 0.05).sum())
83     n_down = int((df['change_pct'] < -0.05).sum())
84     n_flat = len(df) - n_up - n_down
85     return {
86         'n': len(df),
87         'n_up': n_up,
88         'n_down': n_down,
89         'n_flat': n_flat,
90         'avg_pct': float(df['change_pct'].mean()),
91         'total_mcap_T': float(df['market_cap_B'].sum() / 1000), # tỷ → nghìn tỷ
92         'total_value_B': float(df['value_M'].sum() / 1000),    # MM → tỷ
93     }
94
95
96 def sector_overview(df: pd.DataFrame) -> pd.DataFrame:
97     """Gộp theo ngành: avg %, tổng market cap, số mã, advancers/decliners."""
98     if df.empty:
99         return pd.DataFrame()
100     g = df.groupby('sector', dropna=False)
101     out = g.agg(
102         n=('ticker', 'count'),
103         avg_pct=('change_pct', 'mean'),
104         mcap_B=('market_cap_B', 'sum'),
105         value_M=('value_M', 'sum'),
106         n_up=('change_pct', lambda s: int((s > 0.05).sum())),
107         n_down=('change_pct', lambda s: int((s < -0.05).sum()))),
108     ).reset_index()
109     return out.sort_values('mcap_B', ascending=False).reset_index(drop=True)
110
111
112 def top_movers(df: pd.DataFrame, n: int = 5) -> tuple:
113     """(gainers_df, losers_df) – top n theo change_pct."""
114     if df.empty:
115         return pd.DataFrame(), pd.DataFrame()
116     return (df.nlargest(n, 'change_pct').reset_index(drop=True),
117             df.nsmallest(n, 'change_pct').reset_index(drop=True))

```

5. Ghi chú thực thi

Hai chi tiết kỹ thuật đáng chú ý:

1. **Bao contextlib.redirect_stdout/redirect_stderr:** vnstock in các thông báo bằng tiếng Việt ra console (cảnh báo cập nhật, quảng cáo bản Insiders). Trên hệ điều hành Windows với mã hoá cp1252, các ký tự tiếng Việt gây UnicodeEncodeError làm vỡ hàm fetch. Nhóm bọc các lời gọi vnstock trong khối nuốt stdout/stderr để tránh sự cố này (đã quan sát trên mã ACB).
2. **Tier-aware throttle:** hàm throttle() phân biệt hai loại lời gọi – foreground (is_foreground=True) chỉ chờ tối thiểu 1,6 giây; background/warmer chờ 3,4 giây và nhường nhịn khi foreground vừa

claim slot. Cơ chế này giúp khi người dùng đổi mã chứng khoán, ứng dụng không bị treo chờ warmer đã chiếm token trước đó.